

LINQ Aggregate Method in C# with Examples

In this article, I will discuss the **LINQ Aggregate Method in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [LINQ Count Method in C#](#) with Examples. As part of this article, we will discuss the following pointers.

1. [What is the LINQ Aggregate Method in C#?](#)
2. [Example Comma-Separated String](#)
3. [Example Product of Integer Numbers](#)
4. [Aggregate Method With the Seed Parameter](#)
5. [LINQ Aggregate Method with Complex Type in C#](#)
6. [Aggregate Method with Result Selector](#)
7. [When should you use the LINQ Aggregate Method in C#?](#)
8. [When should you not use the LINQ Aggregate Method in C#?](#)

What is the LINQ Aggregate Method in C#?

The Aggregate method in LINQ is used to apply an accumulator function over a sequence. It's part of the System.Linq namespace is typically used to produce a single value from a collection of values. This method can be used for various complex aggregation operations, such as summing values, concatenating strings, or even calculating custom aggregates.

The Aggregate method performs a specified operation on each collection item in a way that carries the result forward. It's like a rolling computation that applies a function to the first two elements, then applies the same function to the result and the next element, and so on, until the entire collection has been processed. There are three overloaded versions of this method available in the **System.Linq** namespace, which is shown in the below image.

```
public static TSource Aggregate<TSource>(this IEnumerable<TSource> source, Func<TSource, TSource, TSource> func);

public static TAccumulate Aggregate<TSource, TAccumulate>(this IEnumerable<TSource> source, TAccumulate seed, Func<TAccumulate, TSource, TAccumulate> func);

public static TResult Aggregate<TSource, TAccumulate, TResult>(this IEnumerable<TSource> source, TAccumulate seed, Func<TAccumulate, TSource, TAccumulate> func, Func<TAccumulate, TResult> resultSelector);
```

The key to effectively using Aggregate lies in understanding how the accumulator function works. The function takes two parameters: the accumulated result so far and the next element of the collection. The function's return value becomes the accumulated result for the next iteration. This continues until all elements have been processed and the final accumulated result is returned. Let us understand the use of the Aggregate method with some examples.

Example: Comma-Separated String.

Let us consider we have the following string array

```
string[] skills = { "C#.NET", "MVC", "WCF", "SQL", "LINQ", "ASP.NET"};
```

Our requirement is to combine all the above strings in the skill array into a single comma-separated string, as shown below.

C#.NET, MVC, WCF, SQL, LINQ, ASP.NET

Program without using the LINQ Aggregate method:

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] skills = { "C#.NET", "MVC", "WCF", "SQL", "LINQ", "ASP.NET" };
            string result = string.Empty;
            foreach(string skill in skills)
            {
                result = result + skill + ", ";
            }
            //Find the index position of last comma
            int lastIndex = result.LastIndexOf(",");
            //Remove the last comma
            result = result.Remove(lastIndex);
            Console.WriteLine(result);
            Console.ReadKey();
        }
    }
}
```

Output:

```
C#.NET, MVC, WCF, SQL, LINQ, ASP.NET
```

Let us see how to achieve the same output using the Linq Aggregate method.

Program using LINQ Aggregate Method in C#:

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] skills = { "C#.NET", "MVC", "WCF", "SQL", "LINQ", "ASP.NET" };
            string result = skills.Aggregate((s1, s2) => s1 + ", " + s2);
        }
    }
}
```

```

        Console.WriteLine(result);
        Console.ReadKey();
    }
}

```

Output:

```
C#.NET, MVC, WCF, SQL, LINQ, ASP.NET
```

How Does the Above LINQ Aggregate Method Work?

The lambda expression $(s1, s2) \Rightarrow s1 + ", " + s2$ will be treated like $s1 = s1 + ", " + s2$, where $s1$ will be accumulated for each item present in the collection. As a result, the Aggregate function will return the comma-separated string. Please look at how the comma-separated string is generated step by step.

1. **Step 1:** First, "C#.NET" is concatenated with "MVC" to produce the result "C#.NET, MVC."
2. **Step 2:** The result in Step 1, i.e., "C#.NET, MVC," is then concatenated with "WCF" to produce the result "C#.NET, MVC, WCF."
3. **Step 3:** The result in Step 2, i.e., "C#.NET, MVC, WCF," is then concatenated with "SQL" to produce the result "C#.NET, MVC, WCF, SQL."
4. **Step 4:** The result in Step 3, i.e., "C#.NET, MVC, WCF, SQL," is then concatenated with "LINQ" to produce the result "C#.NET, MVC, WCF, SQL, LINQ."
5. **Step 5:** The result in Step 4, i.e., "C#.NET, MVC, WCF, SQL, LINQ," is then concatenated with "ASP.NET" to produce the final result "C#.NET, MVC, WCF, SQL, LINQ, ASP.NET" what we see in the output.

Example: Product of Integer Numbers

Consider we have the following integer array

```
int[] intNumbers = { 3, 5, 7, 9 };
```

Our requirement is to compute the product of all numbers.

Program Without using LINQ Aggregate Method in C#:

```

namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = { 3, 5, 7, 9 };
            int result = 1;
            foreach(int num in intNumbers)
            {
                result = result * num;
            }
        }
    }
}

```

```

        Console.WriteLine(result);
        Console.ReadKey();
    }
}

```

Program using LINQ Aggregate Method in C#:

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = { 3, 5, 7, 9 };
            int result = intNumbers.Aggregate((n1, n2) => n1 * n2);
            Console.WriteLine(result);
            Console.ReadKey();
        }
    }
}

```

How does the above Aggregate Method work?

1. **Step 1:** First, it multiplies (3X5) to produce the result of 15
2. **Step 2:** The result of Step 1, i.e., 15, is then multiplied by 7 to produce the result as 105
3. **Step 3:** The result of Step 2, i.e., 105, is multiplied by 9 to produce the final result as 945.

Aggregate Method With the Seed Parameter:

The second overloaded version of the Aggregate method takes the first parameter as the seed value to accumulate. The Second parameter is Func type delegate: Let us understand the use of the seed parameter with an example. Let us see how to pass the seed value as 2 with our previous example.

```
int result = intNumbers.Aggregate(2, (n1, n2) => n1 * n2);
```

The complete example is given below.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {

```

```

        int[] intNumbers = { 3, 5, 7, 9 };
        int result = intNumbers.Aggregate(2, (n1, n2) => n1 * n2);
        Console.WriteLine(result);
        Console.ReadKey();
    }
}

```

Output: 1890

How Does It Work?

1. **Step 1:** First, it multiplies (2*3) to produce the result as 6
2. **Step 2:** The result of Step 1, i.e., 6, is then multiplied by 5 to produce the result as 30
3. **Step 3:** The result of Step 2, i.e., 30, is then multiplied by 7 to produce the result as 210.
4. **Step 4:** The result of Step 3, i.e., 210, is then multiplied by 9 to produce the final result as 1890.

The Aggregate method can be considered a general-purpose tool for producing a single, cumulative result from a sequence. It's a powerful function that can be used for a variety of tasks beyond just summing numbers, including but not limited to:

- Multiplying all elements together to get a product.
- Combining strings or building a custom string representation of a sequence.
- Determining minimum or maximum values under custom rules.
- Accumulating a complex object, like merging a list of objects into a single summary object.

LINQ Aggregate Method with Complex Type in C#:

We are going to work with the following Employee class.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            List<Employee> listStudents = new List<Employee>()
            {
                new Employee{ID= 101,Name = "Preety", Salary = 10000, Department = "Sales"},
                new Employee{ID= 102,Name = "Priyanka", Salary = 15000, Department = "Sales"},
                new Employee{ID= 103,Name = "James", Salary = 50000, Department = "Sales"},
                new Employee{ID= 104,Name = "Hina", Salary = 20000, Department = "Sales"}
            }
        }
    }
}

```

```

        new Employee{ID= 105, Name = "Anurag", Salary = 30000, Department = "IT"}
    };
    return listStudents;
}
}
}

```

While working with the complex type, we need to use either the second or third overloaded version of the Aggregate method. The following example uses the Aggregate method to add the salary of all the employees.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Salary = Employee.GetAllEmployees()
                .Aggregate<Employee, int>(0,
                    (TotalSalary, emp) => TotalSalary += emp.Salary);

            Console.WriteLine(Salary);
            Console.ReadKey();
        }
    }
}

```

Please note here we passed the seed value as 0. Once you run the application, it gives you the output as expected. In the following example, we pass a string as the seed value to the Aggregate extension method. Here, the seed value is “Employee Names”.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string CommaSeparatedEmployeeNames =
Employee.GetAllEmployees().Aggregate<Employee, string>(
                "Employee Names: ", // seed value
                (employeeNames, employee) => employeeNames + employee.Name + ", ");
            int LastIndex = CommaSeparatedEmployeeNames.LastIndexOf(",");
            CommaSeparatedEmployeeNames = CommaSeparatedEmployeeNames.Remove(LastIndex);
            Console.WriteLine(CommaSeparatedEmployeeNames);
            Console.ReadKey();
        }
    }
}

```

```

    }
}

```

Output: Employee Names: Preety, Priyanka, James, Hina, Anurag

In the above example, the first parameter of the Aggregate method is the “**Employee Names:**” string that will be accumulated with all employee names. The comma in the lambda expression will be passed as the second parameter.

Aggregate Method with Result Selector

The third overload version requires the third parameter of the Func delegate expression for the result selector to formulate the result. In our previous example, once we get the comma-separated string, we remove the last comma using some additional logic. Let us see how we can do the same using the third parameter.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string CommaSeparatedEmployeeNames =
Employee.GetAllEmployees().Aggregate<Employee, string, string>(
                "Employee Names: ", // seed value
                (employeeNames, employee) => employeeNames + employee.Name + ", ",
                employeeNames => employeeNames.Substring(
                    employeeNames.Length - 1));

            Console.WriteLine(CommaSeparatedEmployeeNames);
            Console.ReadKey();
        }
    }
}

```

Output: Employee Names: Preety, Priyanka, James, Hina, Anurag

In the above example, we have specified a lambda expression, i.e., **employeeNames => employeeNames.Substring(0, employeeNames.Length – 1)** to remove the last comma in the string result.

When should you use the LINQ Aggregate Method in C#?

Here are some situations where Aggregate is particularly appropriate:

- **Cumulative or Sequential Operations:** When you need to perform operations that require the result of the previous operation. For example, calculating the product of all numbers in a collection, where

each multiplication depends on the result of the previous one.

- **Custom Aggregation:** If the standard aggregation methods like Sum, Max, Min, and Average do not fit your needs, Aggregate allows you to define a custom aggregation logic. For instance, you can use Aggregate to concatenate strings in a specific manner.
- **Handling Complex Data:** Aggregate can be used to combine complex data types or perform operations that involve transforming a collection into a different form. For example, you could use Aggregate to merge a collection of custom objects into a single summary object.
- **Seed Value Operations:** Aggregate provides an overload that allows you to specify a seed value. This is useful when the initial value for the accumulation is not the first element of the collection or when you need a specific type of result.
- **Performance Optimization:** In some cases, using Aggregate can be more efficient than equivalent operations using multiple LINQ methods because it can reduce the need for multiple iterations over the data.
- **Creating Hierarchical or Nested Structures:** Aggregate can be used to build up hierarchical or nested structures from a flat collection, especially when the accumulation process involves nesting or grouping elements based on certain conditions.

In the next article, I will discuss the [Quantifier Operators in LINQ](#) with examples. In this article, I explain the **LINQ Aggregate Method in C#** with Examples, and I hope you understand the need and use of the LINQ Aggregate Method in C#.